

CA2 - Supervised machine learning classification pipeline - applied to pumpkin seed variety classification

By Brage Bromset Bestvold and Lasse Lindholm

Important information

- Do **not** use scikit-learn (`sklearn`) or any other high-level machine learning library for this CA
- Explain your code and reasoning in markdown cells or code comments
- Label all graphs and charts if applicable
- If you use code from the internet, make sure to reference it and explain it in your own words
- If you use additional function arguments, make sure to explain them in your own words
- Use the classes `Perceptron` , `Adaline` and `Logistic Regression` from the library `mlxtend` as classifiers (`from mlxtend.classifier import Perceptron, Adaline, LogisticRegression`). Always use the argument `minibatches=1` when instantiating an `Adaline` or `LogisticRegression` object. This makes the model use the gradient descent algorithm for training. Always use the `random_seed=42` argument when instantiating the classifiers. This will make your results reproducible.
- You can use any plotting library you want (e.g. `matplotlib` , `seaborn` , `plotly` , etc.)
- Use explanatory variable names (e.g. `X_train` and `X_train_scaled` for the training data before and after scaling, respectively)
- The dataset is provided in the file `pumpkin_dataset.csv`

About the dataset

Each row describes one pumpkin seed, measured automatically from a photograph. The four features are geometric properties of the seed:

- `Compactness` : how close the seed's shape is to a circle (ratio; 1 = perfectly round, smaller = more elongated)
- `Major_Axis_Length` : length of the longest line that fits inside the seed (pixels)
- `Extent` : ratio of the seed area to the area of its bounding box
- `Area` : number of pixels inside the seed

The target `class` is the seed variety: `0` = Çerçevelik, `1` = Ürgüp Sivrisi.

The data comes from Koklu, Sarigil & Ozbek (2021), *The use of machine learning methods in classification of pumpkin seeds (Cucurbita pepo L.)*, Genetic Resources and Crop Evolution 68(7), 2713–2726. <https://doi.org/10.1007/s10722-021-01226-0>

Additional clues

- Use the `pandas` library for initial data inspection and preprocessing
- Before training the classifiers, convert the data to raw `numpy` arrays
- For Part IV, you are aiming to create a plot that looks similar to this:



Additional information

- Feel free to create additional code or markdown cells if you think it will help you explain your reasoning or structure your code (you don't have to).

Part I: Data loading and data exploration

Import necessary libraries/modules:

```
In [88]: # Insert your code below
# =====
from mlxtend.classifier import Perceptron, Adaline, LogisticRegression
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Loading and exploring data

1. Load the dataset `pumpkin_dataset.csv` with `pandas` .
2. Check for missing data, report your findings, and remove samples with missing data if any are present.
3. Display the raw data with appropriate plots/outputs and inspect it. Describe the distributions of:
 - feature `"Compactness"`
 - feature `"Area"`
 - the target variable `"class"`
4. Will it be beneficial to scale the data? Why or why not?
5. Is the data linearly separable using any pairwise combination of features? Based on your observations, can we expect an accuracy close to 100% from a linear classifier?

```
In [89]: # Insert your code below
# =====
data = pd.read_csv("pumpkin_dataset.csv")

# Check for missing data
print(data.shape)
data = data.dropna()
print(data.shape)

print(data.columns)

print("""*61)
print("Since no rows have been removed there isn't any data missing")
print("""*61)

#plot the raw data
fig, ax = plt.subplots(1, 5, figsize=(15,5))

ax[0].violinplot(data["Compactness"])
ax[0].set_title("Distribution of Compactness")

ax[1].violinplot(data["Major_Axis_Length"])
ax[1].set_title("Distribution of Axis Length")

ax[2].violinplot(data["Extent"])
ax[2].set_title("Distribution of Extent")

ax[3].violinplot(data["Area"])
ax[3].set_title("Distribution of Area")

ax[4].bar(
    data["class"].value_counts().sort_index().index,
    data["class"].value_counts().sort_index().values
)

ax[4].set_title("Class distribution")
ax[4].set_xticks([0, 1])

plt.show()
```

```
sns.pairplot(
    data,
    vars=["Compactness", "Major_Axis_Length", "Extent", "Area"],
    hue="class"
)

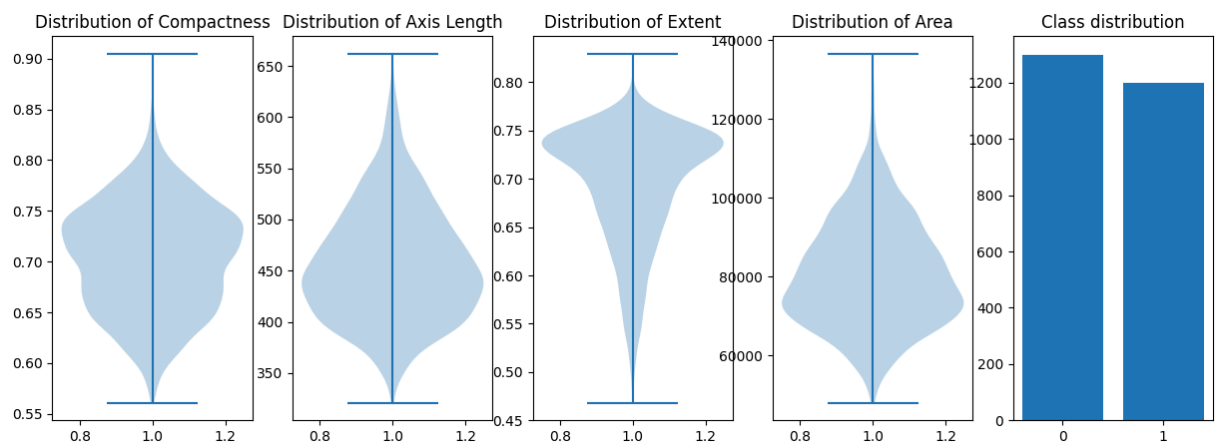
plt.show()
```

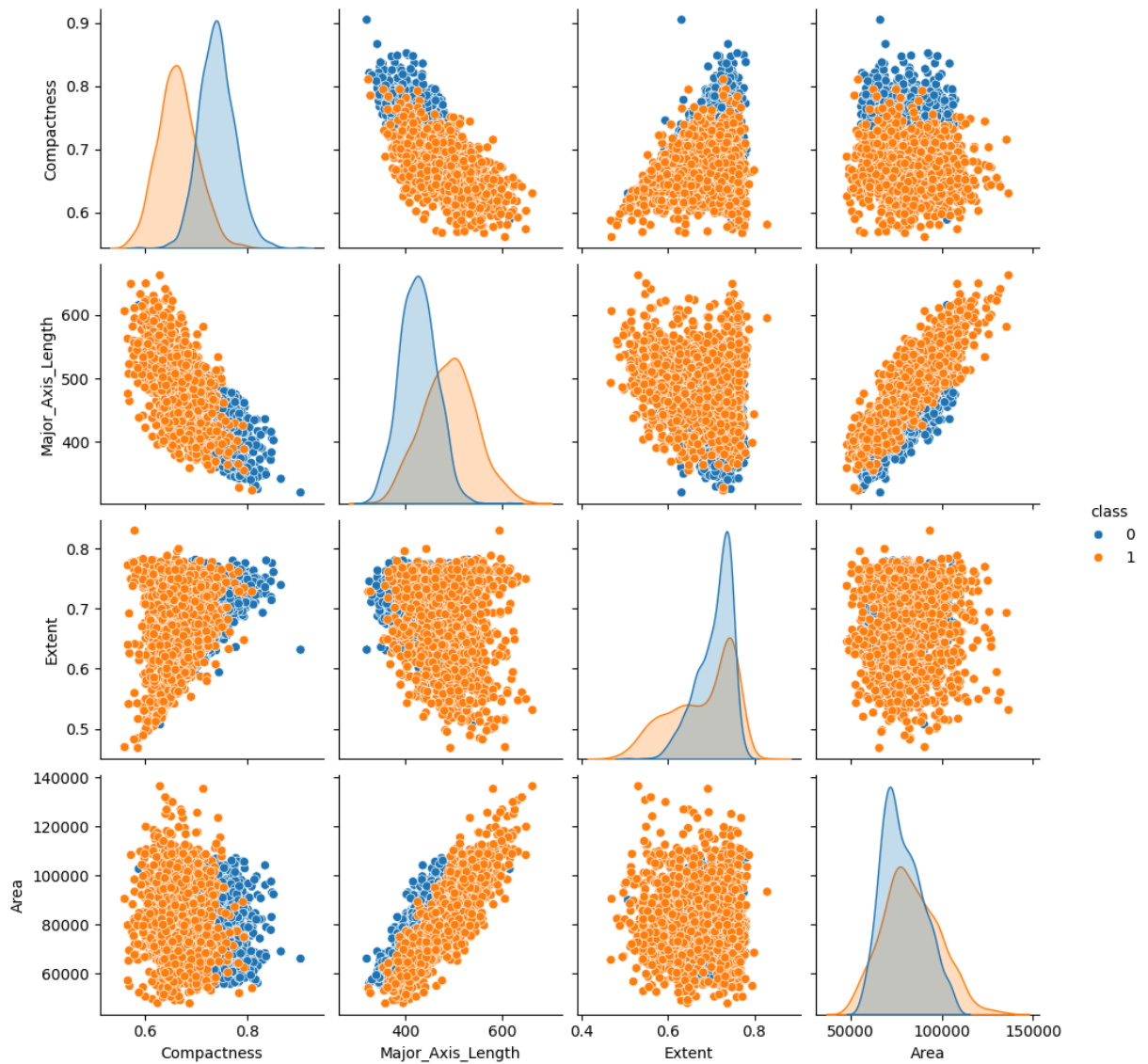
(2500, 5)

(2500, 5)

Index(['Compactness', 'Major_Axis_Length', 'Extent', 'Area', 'class'], dtype='object')

Since no rows have been removed there isn't any data missing





Compactness

The distribution of compactness is fairly good, and the distribution have a peak at around 0.74

Area

The distribution of area is right-skewed with most entries on the lower side of its scale and a long tail upward, the distribution peaks at around 75 000

class

There are two classes, with class 0 having slightly more entries than class 1

Scaling

It will be highly beneficial to scale the data, since the compactness uses values between 0 and 1 while the area uses value going up towards 140 000

Linearly seperable

Based on the pairplots the data does not seem to be linearly seperable, so we do not expect to get 100% accuracy

Part II: Train/Test Split

Divide your dataset into training and testing subsets. Follow these steps to create the split:

1. Divide the dataset into two data sets, each containing samples of only one class:

- Create a DataFrame `df_0` containing all samples with `"class"` equal to `0`.
- Create a DataFrame `df_1` containing all samples with `"class"` equal to `1`.

2. Split into training and test sets by randomly sampling entries from the data frames:

- Create a DataFrame `df_0_train` by sampling `75%` of the entries from `df_0` (use the `sample` method with `random_state=42`).
- Create a DataFrame `df_1_train` using the same approach with `df_1`.
- Create a DataFrame `df_0_test` containing the remaining entries of `df_0` (use `df_0.drop(df_0_train.index)`).
- Create a DataFrame `df_1_test` using the same approach with `df_1`.

3. Merge the datasets split by classes back together:

- Create a DataFrame `df_train` containing all entries from `df_0_train` and `df_1_train` (use the `concat` method).
- Create a DataFrame `df_test` containing all entries from the two test sets.

4. Create the following data frames from these splits:

- `X_train` : contains all columns of `df_train` except for the target feature `"class"`
- `X_test` : contains all columns of `df_test` except for the target feature `"class"`
- `y_train` : contains only the target feature `"class"` for all samples in the training set
- `y_test` : contains only the target feature `"class"` for all samples in the test set

5. Check that your sets have the expected sizes/shape by printing the number of rows and columns (`shape`) of the data sets.

- (Sanity check: there should be **4 features**, and the training set should contain roughly **75%** of the total samples, with the test set containing the remaining **25%**.)

6. **Explain the purpose of this slightly complicated procedure.**

- Why did we first split into the two classes?
- Why did we then split into a training and a testing set?

7. **What is the share (in percent) of samples with class 0 in the test set, training set, and in the initial data set?**

```
In [90]: # Insert your code below
# =====
"""
=====
    task description
=====
create df0 and df1
create df 0 and 1 train
create df 0 and 1 test

create df_train (concat 0 and 1)
create df test with all entries from test

X_train, X_test, y_train, y_test

Check for expected sizes and shapes, should contain 75% of total samples with the r

Explain why split it into two classes, why a training set and a testing set
"""

# part 1,
# creates a dataframe from the csv

# then uses that dataframe to create two new ones, each only containing pumpkins wh
df_0 = data[data["class"] == 0]
df_1 = data[data["class"] == 1]

# part 2,
# create df_0_train from 75% of entries, df_1_train from 75% of entries, use sample
# and df_0_test and df_1_test from the remaining 25% in respective origin datafra

df_0_train = df_0.sample(frac=0.75, random_state=42) # use 75% of df_0, with random
# = 42, (the answer to the ultimate question of life, the universe and everything)

df_1_train = df_1.sample(frac=0.75, random_state=42) # same as before, random state

# drop the indexes contained in the training data
df_0_test = df_0.drop(df_0_train.index)
df_1_test = df_1.drop(df_1_train.index)

# part 3,
# merge them back together using concat
df_train = pd.concat([df_0_train, df_1_train], ignore_index=True, sort=False)
```

```

df_test = pd.concat([df_0_test, df_1_test], ignore_index=True, sort=False)

# part 4,
# create the X_train, X_test, y_train, y_test

# create X train and test , and exclude the column "class"
X_train = df_train.drop(columns=["class"]).copy()
X_test = df_test.drop(columns=["class"]).copy()

# the opposite, y_train and test should ONLY have "class"
y_train = df_train["class"]
y_test = df_test["class"]

# part 5,
# sanity check with shape
print(f"X_train: {X_train.shape}")
print(f"X_test: {X_test.shape}")

print(f"y_train: {y_train.shape}")
print(f"y_test: {y_test.shape}")

# part 6
# explain why divide it like this:
# there are 2 types of pumpkins and we separate them to keep the same ratio when tr

# They are split into training and testing sets because if you test the model on da

# part 7
# what is the share with class = 0, for all data, training, and testing?
data_tot = len(data)
data_0 = (len(df_0) / len(data)) * 100
print(f"percentage in all of data is 0: {data_0}%")

train_tot = len(df_0_train) + len(df_1_train)
train_0 = (len(df_0_train) / train_tot) * 100
print(f"percentage for train 0: {train_0}%")

test_tot = len(df_0_test) + len(df_1_test)
test_0 = (len(df_0_test) / test_tot) * 100
print(f"percentage for test 0: {test_0}%")

```

```

X_train: (1875, 4)
X_test: (625, 4)
y_train: (1875,)
y_test: (625,)
percentage in all of data is 0: 52.0%
percentage for train 0: 52.0%
percentage for test 0: 52.0%

```

Convert data to numpy arrays and shuffle the training data

Many machine learning models (including those you will use later in this assignment) do not accept `pandas` DataFrames as input. Instead, they require the data to be provided as raw `numpy` arrays.

In this step, convert the DataFrames `X_train`, `X_test`, `y_train`, and `y_test` into the corresponding numpy arrays with the same variable names.

In addition, shuffle the training data. This is necessary because the training set is currently ordered by class (all samples of class 0 followed by all samples of class 1).

In Part IV, the classifiers will be trained using the **first n samples** of the training set. If the data were not shuffled, the models would initially be trained only on samples from a single class, which would lead to biased learning and unreliable performance.

Nothing to be done here, just execute the cell.

```
In [91]: # convert to numpy arrays
X_train = X_train.to_numpy()
X_test = X_test.to_numpy()
y_train = y_train.to_numpy()
y_test = y_test.to_numpy()

# shuffle training data
np.random.seed(42) # for reproducibility
shuffle_index = np.random.permutation(len(X_train)) # generate random indices
X_train, y_train = X_train[shuffle_index], y_train[shuffle_index] # shuffle data by
```

Part III: Scaling the data

1. Standardize the training *and* test data so that each feature has a mean of 0 and a standard deviation of 1.
2. Check that the scaling was successful
 - by printing the mean and standard deviation of each feature in the scaled training set
 - by putting the scaled training set into a DataFrame and making a violin plot of the data

Hint: use the `axis` argument to calculate mean and standard deviation column-wise.

Important: Avoid data leakage!

Compute the mean and standard deviation only on the training data, and then apply the same transformation to both the training and test sets.

More hints:

1. For each column, subtract the training mean (μ_{train}) from each value in that column
2. Divide the result by the training standard deviation (σ_{train})

Mathematically, this transformation for each column is:

$$X_{\text{scaled}} = \frac{X - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

where:

- X_{scaled} are the transformed column values (a column vector)
- X is the original column
- μ_{train} is the mean of the column computed on the training set
- σ_{train} is the standard deviation of the column computed on the training set

```
In [92]: df_0.columns
```

```
Out[92]: Index(['Compactness', 'Major_Axis_Length', 'Extent', 'Area', 'class'], dtype='object')
```

```
In [93]: # Insert your code below
# =====

# Find mean and std of training data
trainMean = X_train.mean(axis=0)
trainStd = X_train.std(axis=0)

# Scale training data
X_train_scaled = X_train - trainMean
X_train_scaled = X_train_scaled / trainStd

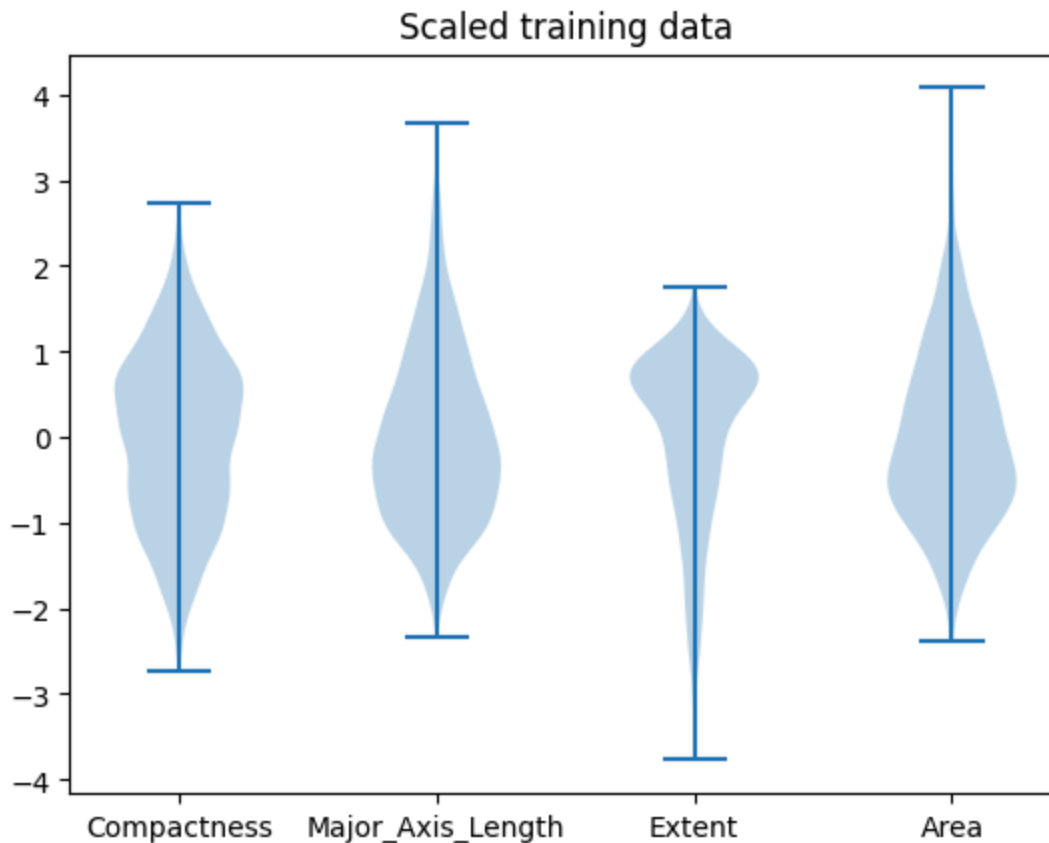
# Scale test data with mean and std from training data
X_test_scaled = X_test - trainMean
X_test_scaled = X_test_scaled / trainStd

print(X_train_scaled.mean(axis=0))

# Plot scaled data as violinplot
dfScaled = pd.DataFrame(X_train_scaled)
dfScaled.columns = df_0.columns.drop("class")

plt.violinplot(dfScaled)
plt.title("Scaled training data")
plt.xticks(range(1, len(dfScaled.columns) + 1), dfScaled.columns)
plt.show()
```

```
[ 1.63539589e-14 -8.46582064e-16 -1.05821130e-14  1.49332398e-16]
```



Part IV: Training and evaluation with different dataset sizes and training times

Often, a larger dataset size will yield better model performance. (As we will learn later, this usually prevents overfitting and increases the generalization capability of the trained model.) However, collecting data is usually rather expensive.

In this part of the exercise, you will investigate

- how the model performance changes with varying dataset size
- how the model performance changes with varying numbers of epochs/iterations of the optimizer/solver (increasing training time).

For this task (Part IV), use the `Adaline`, `Perceptron`, and `LogisticRegression` classifiers from the `mlxtend` library. All use the gradient descent (GD) algorithm for training.

Important: Use a learning rate of `1e-5` (`0.00001`) for all classifiers, and use the argument `minibatches=1` when initializing the `Adaline` and `LogisticRegression` classifiers (this will make sure they use GD). For all three classifiers, pass `random_seed=42` when initializing the classifier to ensure reproducibility of the results.

Model training

Train the models using progressively larger subsets of your dataset, specifically: first 50 rows, first 100 rows, first 150 rows, ..., first 650 rows, first 700 rows (in total 14 different variants).

For each number of rows train the model with progressively larger number of epochs: 2, 7, 12, 17, ..., 87, 92, 97 (in total 20 different model variants).

This will result in $14 \times 20 = 280$ models obtained from the different combinations of subsets and number of epochs. An output of the training process could look like this:

```
Model (1) Train a model with first 50 rows of data for 2 epochs
Model (2) Train a model with first 50 rows of data for 7 epochs
Model (3) Train a model with first 50 rows of data for 12 epochs
...
Model (21) Train a model with first 100 rows of data for 2 epochs
Model (22) Train a model with first 100 rows of data for 7 epochs
...
Model (279) Train a model with first 700 rows of data for 92 epochs
Model (280) Train a model with first 700 rows of data for 97 epochs
```

Model evaluation

For each of the 280 models, calculate the **accuracy on the test set** (do **not** use the score method but compute accuracy yourself). Store the results in the provided 2D numpy array (it has 14 rows and 20 columns). The rows of the array correspond to the different dataset sizes, and the columns correspond to the different numbers of epochs.

Tasks

1. Train the 280 Adaline classifiers as mentioned above and calculate the accuracy for each of the 280 variants.
2. Generalize your code so that it is doing the same procedure for all three classifiers: `Perceptron`, `Adaline`, and `LogisticRegression` after each other. Store the result for all classifiers. You can for example use an array of shape $3 \times 14 \times 20$ to store the accuracies of the three classifiers.

Note that executing the cells will take some time (but on most systems it should not be more than 5 minutes).

```
In [94]: # Train and evaluate all model variants
# Insert your code below
# =====

"""
for this part of the task, use Adaline, Perceptron and LogisticRegression from mlxt
use gradient descent algorithm for training.
learning rate should be 1e-5,

Train the models using progressively larger subsets of your dataset, specifically:
```

first 50 rows, first 100 rows, first 150 rows, ..., first 650 rows, first 700 rows

For each number of rows train the model with progressively larger number of epochs:
2, 7, 12, 17, ..., 87, 92, 97 (in total 20 different model variants).

This will result in $14 \times 20 = 280$ models obtained from the different combinations.
An output of the training process could look like this:

```
'''
Model (1) Train a model with first 50 rows of data for 2 epochs
Model (2) Train a model with first 50 rows of data for 7 epochs
Model (3) Train a model with first 50 rows of data for 12 epochs
...
Model (21) Train a model with first 100 rows of data for 2 epochs
Model (22) Train a model with first 100 rows of data for 7 epochs
...
Model (279) Train a model with first 700 rows of data for 92 epochs
Model (280) Train a model with first 700 rows of data for 97 epochs
'''

"""

# should include the 14 values ranging from 50 to 700
dataset_sizes = [i for i in range(50, 701, 50)]
print(dataset_sizes)
print(len(dataset_sizes))

# should contain the 20 different epoch lengths ranging from 2 to 97
epoch_counts = [i for i in range(2, 98, 5)]
print(epoch_counts)
print(len(epoch_counts))

classifiers = {"Perceptron":Perceptron, "Adaline":Adaline, "LogisticRegression":LogisticRegression}

results = np.zeros(shape=(3, 14, 20)) # classifier, index, size

for index, clf in enumerate(classifiers):
    counter = 0
    for size, n_rows in enumerate(dataset_sizes):
        X_subset = X_train_scaled[:n_rows]
        y_subset = y_train[:n_rows]

        for epoch_in, n_epochs in enumerate(epoch_counts):
            counter += 1
            # print information for this epoch
            print(f"Model ({counter}) Train a {clf} with first {n_rows} rows for {n_epochs} epochs")

            # Train the 3 different models in the classifiers dict
            # with the random seed 42, just like the answer to the ultimate question
            if clf == "Perceptron":
                model = Perceptron(eta=1e-5, epochs=n_epochs, random_seed=42)
            else:
                model = classifiers[clf](eta=1e-5, epochs=n_epochs, minibatches=1,
```

```
# fit the model with the current model
model.fit(X_subset, y_subset)

# use fitted model to predict, test accuracy, then save to results
y_pred = model.predict(X_test_scaled)
accuracy = np.mean(y_pred == y_test)
results[index, size, epoch_in] = accuracy
```

```
[50, 100, 150, 200, 250, 300, 350, 400, 450, 500, 550, 600, 650, 700]
```

```
14
```

```
[2, 7, 12, 17, 22, 27, 32, 37, 42, 47, 52, 57, 62, 67, 72, 77, 82, 87, 92, 97]
```

```
20
```

Model (1) Train a Perceptron with first 50 rows for 2 epochs

Model (2) Train a Perceptron with first 50 rows for 7 epochs

Model (3) Train a Perceptron with first 50 rows for 12 epochs

Model (4) Train a Perceptron with first 50 rows for 17 epochs

Model (5) Train a Perceptron with first 50 rows for 22 epochs

Model (6) Train a Perceptron with first 50 rows for 27 epochs

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

Model (231) Train a LogisticRegression with first 600 rows for 52 epochs
Model (232) Train a LogisticRegression with first 600 rows for 57 epochs
Model (233) Train a LogisticRegression with first 600 rows for 62 epochs
Model (234) Train a LogisticRegression with first 600 rows for 67 epochs
Model (235) Train a LogisticRegression with first 600 rows for 72 epochs
Model (236) Train a LogisticRegression with first 600 rows for 77 epochs
Model (237) Train a LogisticRegression with first 600 rows for 82 epochs
Model (238) Train a LogisticRegression with first 600 rows for 87 epochs
Model (239) Train a LogisticRegression with first 600 rows for 92 epochs
Model (240) Train a LogisticRegression with first 600 rows for 97 epochs
Model (241) Train a LogisticRegression with first 650 rows for 2 epochs
Model (242) Train a LogisticRegression with first 650 rows for 7 epochs
Model (243) Train a LogisticRegression with first 650 rows for 12 epochs
Model (244) Train a LogisticRegression with first 650 rows for 17 epochs
Model (245) Train a LogisticRegression with first 650 rows for 22 epochs
Model (246) Train a LogisticRegression with first 650 rows for 27 epochs
Model (247) Train a LogisticRegression with first 650 rows for 32 epochs
Model (248) Train a LogisticRegression with first 650 rows for 37 epochs
Model (249) Train a LogisticRegression with first 650 rows for 42 epochs
Model (250) Train a LogisticRegression with first 650 rows for 47 epochs
Model (251) Train a LogisticRegression with first 650 rows for 52 epochs
Model (252) Train a LogisticRegression with first 650 rows for 57 epochs
Model (253) Train a LogisticRegression with first 650 rows for 62 epochs
Model (254) Train a LogisticRegression with first 650 rows for 67 epochs
Model (255) Train a LogisticRegression with first 650 rows for 72 epochs
Model (256) Train a LogisticRegression with first 650 rows for 77 epochs
Model (257) Train a LogisticRegression with first 650 rows for 82 epochs
Model (258) Train a LogisticRegression with first 650 rows for 87 epochs
Model (259) Train a LogisticRegression with first 650 rows for 92 epochs
Model (260) Train a LogisticRegression with first 650 rows for 97 epochs
Model (261) Train a LogisticRegression with first 700 rows for 2 epochs
Model (262) Train a LogisticRegression with first 700 rows for 7 epochs
Model (263) Train a LogisticRegression with first 700 rows for 12 epochs
Model (264) Train a LogisticRegression with first 700 rows for 17 epochs
Model (265) Train a LogisticRegression with first 700 rows for 22 epochs
Model (266) Train a LogisticRegression with first 700 rows for 27 epochs
Model (267) Train a LogisticRegression with first 700 rows for 32 epochs
Model (268) Train a LogisticRegression with first 700 rows for 37 epochs
Model (269) Train a LogisticRegression with first 700 rows for 42 epochs
Model (270) Train a LogisticRegression with first 700 rows for 47 epochs
Model (271) Train a LogisticRegression with first 700 rows for 52 epochs
Model (272) Train a LogisticRegression with first 700 rows for 57 epochs
Model (273) Train a LogisticRegression with first 700 rows for 62 epochs
Model (274) Train a LogisticRegression with first 700 rows for 67 epochs
Model (275) Train a LogisticRegression with first 700 rows for 72 epochs
Model (276) Train a LogisticRegression with first 700 rows for 77 epochs
Model (277) Train a LogisticRegression with first 700 rows for 82 epochs
Model (278) Train a LogisticRegression with first 700 rows for 87 epochs
Model (279) Train a LogisticRegression with first 700 rows for 92 epochs
Model (280) Train a LogisticRegression with first 700 rows for 97 epochs

Performance visualization

Plot the performance measure for all classifiers (accuracy on the test set; use the result array from above) of all the 280 variants for each classifier in a total of three heatmaps using, for

example `seaborn` or `matplotlib` directly.

The color should represent the accuracy on the test set, and the x and y axes should represent the number of epochs and the dataset size, respectively. Which one is x and which one is y is up to you to decide. Look in the example output at the top of the assignment for inspiration for how the plot could look like and how it could be labeled nicely. (But use the correct numbers corresponding to your dataset sizes and number of epochs.)

```
In [95]: # Insert your code below
# =====

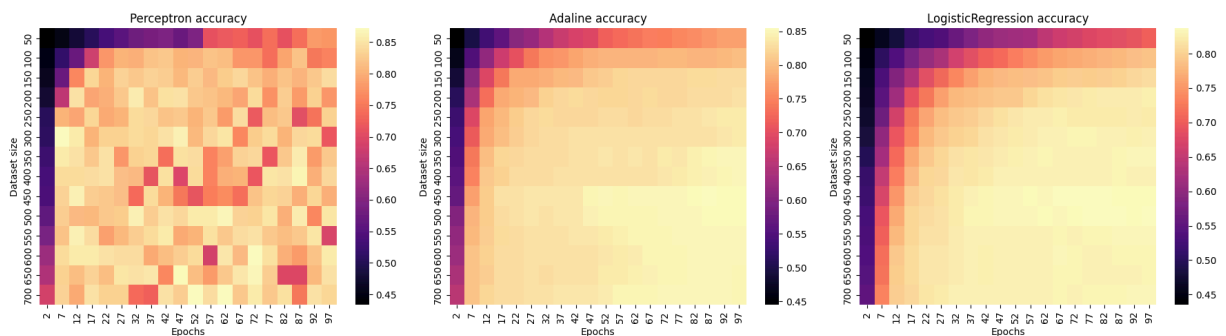
# create a subplot for each model
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# iterate over the results from each model and plot them one by one
for idx, clf in enumerate(classifiers.keys()):
    accuracy_grid = results[idx]
    ax = axes[idx]

    # create the heatmap using seaborn before betraying them and plotting with matplotlib
    heatmap = sns.heatmap(accuracy_grid,
                           xticklabels=epoch_counts,
                           yticklabels=dataset_sizes,
                           cmap="magma",
                           ax=ax,
                           cbar=True
                           )

    # titles
    ax.set_xlabel("Epochs")
    ax.set_ylabel("Dataset size")
    ax.set_title(f"{clf} accuracy")

plt.tight_layout()
plt.show()
```



Part V: Some more plotting

For the following cell to execute you need to have the variables `X_train_scaled` and `y_train` (to train the model) as well as `X_test_scaled` and `y_test` (to plot the test

samples). Complete at least up until Part III. Executing the cell will plot something.

This dataset has 4 features, so the plot will be a 4×4 grid showing pairwise feature decision regions.

1. Add code comments explaining what the lines are doing
2. What is the purpose of the plot?
3. Describe all components of the subplot and then comment in general on the entire plot.
What does it show? What does it not show?

```
In [96]: # The classifier is the logistic regression model, trained for 300 epochs with the
# to the answer of the ultimate question of life, the universe and everything and e
clf = LogisticRegression(eta=1e-5, epochs=300, minibatches=1, random_seed=42)
clf.fit(X_train_scaled, y_train)

# the number of columns
n_features = X_test_scaled.shape[1]

# create subplots in an n x n sized grid
fig, axes = plt.subplots(n_features, n_features, figsize=(5*n_features, 5*n_features))

# get the minimum and the maximum values of the scaled test values
# They set the bounds for the grid
mins = X_test_scaled.min(axis=0)
maxs = X_test_scaled.max(axis=0)

# iterate over every position on both axes
for i in range(n_features):
    for j in range(n_features):
        # apply a value to the matching position
        feature_1 = i
        feature_2 = j
        ax = axes[i, j]

        # Label each axis
        ax.set_xlabel(f"Feature {feature_1}")
        ax.set_ylabel(f"Feature {feature_2}")

        # Create grid for the two selected features
        x0 = np.linspace(mins[feature_1], maxs[feature_1], 100)
        x1 = np.linspace(mins[feature_2], maxs[feature_2], 100)

        X0, X1 = np.meshgrid(x0, x1)

        # Create full feature matrix with other features set to 0
        X_plot = np.zeros((X0.size, n_features))
        X_plot[:, feature_1] = X0.ravel()
        X_plot[:, feature_2] = X1.ravel()

        # Predict probability of class 1
        y_pred = clf.predict_proba(X_plot)
        Z = y_pred.reshape(X0.shape)

        # Plot probability surface
```

```

ax.pcolor(X0, X1, Z, shading="auto")

# Plot decision boundary (p = 0.5)
ax.contour(X0, X1, Z, levels=[0.5], colors="k")

# Plot test samples
ax.scatter(
    X_test_scaled[y_test == 0, feature_1],
    X_test_scaled[y_test == 0, feature_2],
    color="b", marker="^", s=50, facecolors="none"
)

ax.scatter(
    X_test_scaled[y_test == 1, feature_1],
    X_test_scaled[y_test == 1, feature_2],
    color="y", marker="o", s=50, facecolors="none"
)

```

"""

The purpose of this plot is to show how accurate the model is by attempting to predict in 2D grids and showing the test values to visualize accuracy and if the background is representative of the actual distribution of test markers.

functionally, there are 16 plots in a 4x4 grid

each plot has a gradient and some points separated by shape and color 'blue, ^' and these subplots show a background gradient which represents the model's predicted probability of the points (0's and ^'s) which represent the actual test cases and the black line which represents the model's predicted boundary

If there looks like there is a clear boundary with the black line, and the gradient if they both match the distribution of the ^'s and o's then that means that the model is accurate/good, while if there does not appear to accurately match the test points

They do not show the fact that each plot only contains 2 out of 4 features, meaning basically a 2D slice of a 4D figure, this also means that a boundary that looks accurate appear wrong in practice, if the other two features are far from the average

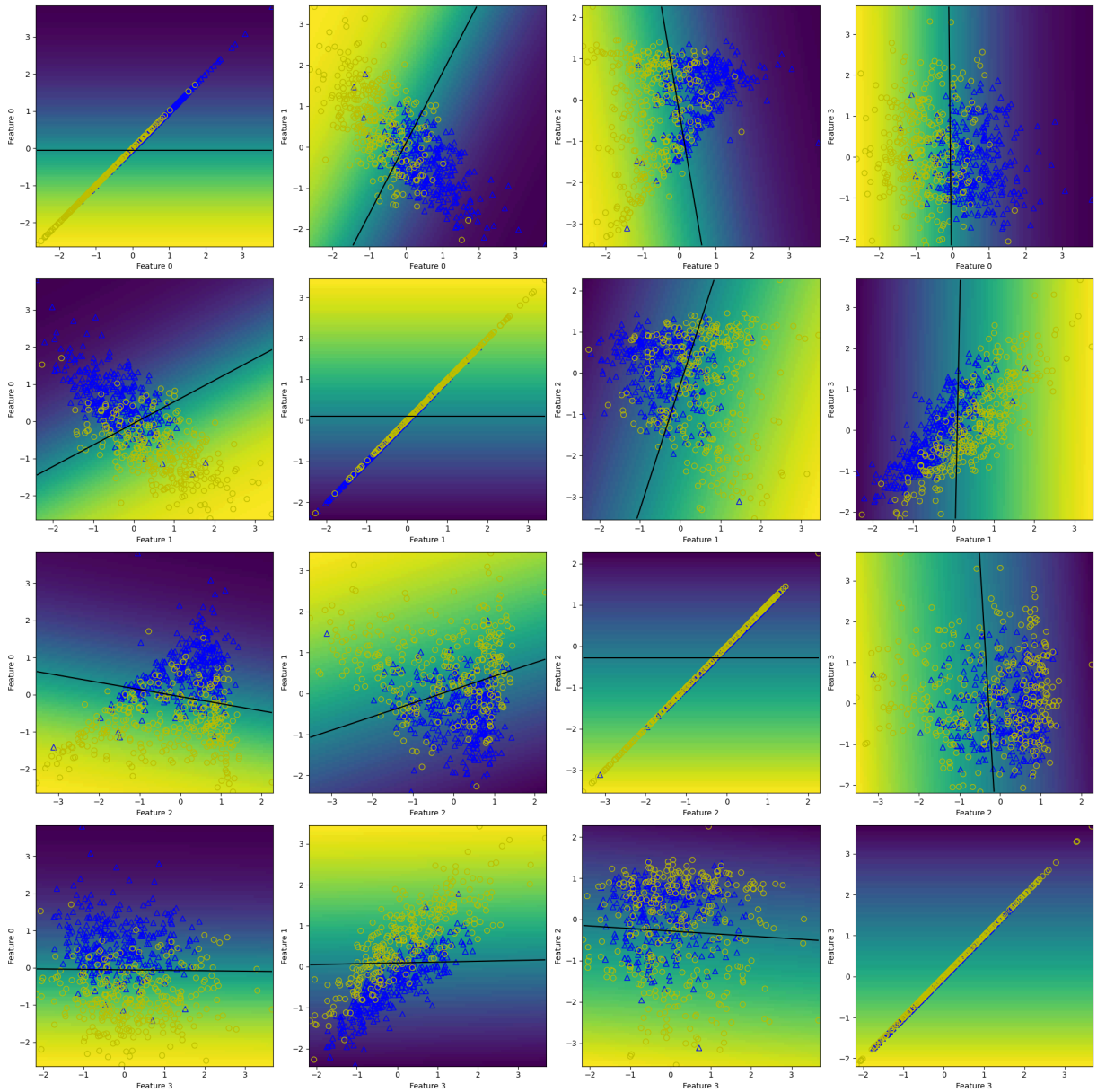
the plots where $i = j$ are not meaningful since they are plotted against itself.

"""

```

# show the plot
fig.tight_layout()
plt.show()

```



Part VI: Additional discussion

Part I:

1. What kind of plots did you use to visualize the raw data, and why did you choose these types of plots?

Part II:

1. What happens if we don't shuffle the training data before training the classifiers like in Part IV?
2. How could you do the same train/test split (Point 1.-4.) using scikit-learn?

Part IV:

1. How does increasing the dataset size affect the performance of the logistic regression model? Provide a summary of your findings.
2. Describe the relationship between the number of epochs and the model accuracy.
3. Which classifier is much slower to train and why do you think that is?
4. One classifier shows strong fluctuations in accuracy for different dataset sizes and number of epochs. Which one is it and why do you think this happens?

Part 1

We visualized the data using violinplots for the 4 features and a barplot for the classes. We used the barplot, because it clearly shows the distribution in the two classes. We chose a violinplot for the other features because it gives an overview over the distribution within each feature.

Part 2

If we don't shuffle the training data before training the classifiers they will receive all the samples from class 0 first and class 1 afterwards. This may give the models a bias towards class 1, since it only re-adjusts according to class 1 samples for the second half of the run. This may lower the accuracy of the classifier.

```
In [97]: # The train/test split can be done using train_test_split from Scikit-Learn
from sklearn.model_selection import train_test_split
xtra, xtes, ytra, ytes = train_test_split(data, data["class"], train_size=0.75, str

# here x and y is respectively the data and the targets (correct class)
# stratify=y makes sure to balance the samples making sure to keep the ratio between
# train_size sets the amount of data from X to be distributed to the training-set
# random_state = 42 makes sure we find the answer to the ultimate question itself)
```

Part 4

For the logistic regression model, increasing the dataset size almost always increases the accuracy of the model. The rate of improvement depends on both the dataset size and the number of epochs. For large numbers of epochs, increasing the size of the dataset increases the accuracy drastically in the lower end of dataset sizes, e.g. from 50 to 100 samples, but this gain quickly dwindles as the model already stops increasing much at 200 samples for the maximum number of epochs. As the number of epochs decreases the number of samples needed to reach this high accuracy increases, with 2-7 epochs showing increased accuracy by increasing dataset size, but never reaching a particularly high value.

The model accuracy increases with increased epochs. Much like for dataset size, the rate of change per increase in epochs is highly dependent on the size of the dataset used. The biggest increase in accuracy is again for low numbers of epochs, for both Adaline and Logistic regression the accuracy already gets high with 12 epochs, given high dataset sizes.

Perceptron fluctuates highly for different numbers of epochs and dataset sizes, but based on the plot, perceptron actually seems to have higher accuracy on lower values of epochs, performing best in the 7-27 range. This is however believed to be due to random chance (the seed) and not representative of an actual pattern. It is also worth noting that having a high number of epochs with a low dataset size performs better than the reverse for all three models.

Perceptron is much slower to train. When testing the model-testing in part 4 by importing the time library, it was found that perceptron was by far the slowest, taking 54.4 seconds to complete, compared to 0.469 for Adaline and 0.717 for Logistic Regression. The huge difference in time is believed to be due to perceptron going through each sample one at a time, while adaline and logistic regression goes through the entire dataset and computes the error for each sample simultaneously.

The model showing high fluctuation in accuracy is Perceptron. This is believed to be because perceptron only updates its weights as it misclassifies a sample, and since the dataset isn't linearly separable, the decision boundary is likely to jump around a lot instead of remaining in the same spot. This is believed to make the accuracy fluctuate highly, since the model is very reactive to the samples it misclassifies.